



Секционированные Индексные Фильтры

[Прыжок на дно](#)

pheesian отредактировал эту страницу 3 февраля 2021 г. · [21 изменение](#)

По мере увеличения соотношения DB/мет объем памяти, занимаемый блоками фильтров/индексов, становится нетривиальным. Хотя `cache_index_and_filter_blocks` позволяет хранить в блочном кэше лишь их часть, их относительно большой размер негативно влияет на производительность, поскольку они i) занимают место в блочном кэше, которое можно было бы использовать для кэширования данных, ii) увеличивают нагрузку на дисковое хранилище, загружая их в кэш после промаха. Здесь мы более подробно рассмотрим эти проблемы и объясним, как секционирование индексов/фильтров снижает нагрузку.

🔗 Какого размера блоки индекса/фильтрации?

По умолчанию в RocksDB на каждый SST-файл приходится один индексный/фильтрационный блок. Размер индексного/фильтрационного блока зависит от конфигурации, но для SST размером 256 МБ типичен размер 0,5–5 МБ, что намного больше, чем типичный размер блока данных — 4–32 КБ. Это нормально, когда все индексные/фильтрационные блоки идеально помещаются в память и, следовательно, считываются один раз за время существования SST. Но не тогда, когда они конкурируют с блоками данных за место в блочном кэше и, скорее всего, многократно перечитываются с диска.

🔗 Что такого особенного в больших блоках индексов/фильтров?

Когда индексные/фильтрационные блоки хранятся в блочном кэше, они фактически конкурируют с блоками данных (а также друг с другом) за этот ограниченный ресурс. Фильтр размером 5 МБ занимает место, которое можно было бы использовать для кэширования тысяч блоков данных (размером 4 КБ). Это приводит к большему количеству промахов в кэше для блоков данных. Кроме того, большие индексные/фильтрационные блоки чаще вытесняют друг друга из блочного кэша и увеличивают количество собственных промахов в кэше. При этом за время пребывания в кэше могла быть использована лишь небольшая часть блока индекса/фильтра.

После промаха в кэше индексный/фильтрационный блок приходится перезагружать с диска, и его большой размер не способствует снижению затрат на ввод-вывод. Хотя для простого поиска по точке может потребоваться максимум пара блоков данных (размером 4 КБ) — по одному с каждого уровня LSM, — в итоге может потребоваться загрузка нескольких мегабайт индексных/фильтрационных блоков. Если это происходит часто, то диск тратит больше времени на обслуживание индексных/фильтрационных блоков, чем на обслуживание самих блоков данных.

🔗 Что такое секционированный индекс/фильтры?

При секционировании индексный/фильтрационный блок SST-файла разбивается на более мелкие блоки с дополнительным индексом верхнего уровня. При чтении индексного/фильтрационного блока в память загружается только индекс верхнего уровня. Затем секционированный индексный/фильтрационный блок использует индекс верхнего уровня для загрузки в кэш блоков тех секций, которые необходимы для выполнения индексного/фильтрационного запроса. Индекс верхнего уровня, занимающий гораздо меньше памяти, может храниться в куче или в кэше блоков в зависимости от параметра `cache_index_and_filter_blocks` .

🔗 Плюсы:

- Более высокая частота попаданий в кэш: вместо того чтобы засорять кэш большими индексами/блоками, секционирование позволяет загружать индексы/фильтры с гораздо большей детализацией и, следовательно, эффективно использовать пространство кэша.
- Меньшая нагрузка на ввод-вывод: при промахе кэша для секции индекса/фильтра с диска требуется загрузить только одну секцию, что значительно снижает нагрузку на диск по сравнению с чтением всего индекса/фильтра из SST-файла.
- Без ущерба для индекса/фильтров: альтернативный подход к уменьшению объема памяти, занимаемого индексом/фильтрами, без секционирования

заключается в том, чтобы пожертвовать их точностью, например за счет увеличения размера блоков данных или уменьшения количества битов Блума, чтобы уменьшить размер индекса и фильтров.

🔗 Минусы:

- Дополнительное пространство для индекса верхнего уровня: оно довольно мало — 0,1–1 % от размера индекса/фильтра.
- Больше операций ввода-вывода на диске: если индекс верхнего уровня еще не в кэше, это приведет к одной дополнительной операции ввода-вывода. Чтобы этого избежать, их можно хранить либо в куче, либо в кэше с высоким приоритетом (TODO: доработать)
- Потеря пространственной локальности: если рабочая нагрузка требует частого, но произвольного чтения из одного и того же SST-файла, это приведет к загрузке отдельного раздела индекса/фильтра при каждом чтении, что менее эффективно, чем чтение всего индекса/фильтра сразу. Хотя мы не наблюдали такой закономерности в наших тестах, она вероятна только для уровней L0/L1 LSM, для которых можно отключить секционирование (TODO: доработать)

🔗 Истории успеха

🔗 Жесткий диск, 100 ТБ

В этом примере у нас есть база данных размером 86 ГБ на жестком диске, и мы имитируем небольшую память узла с 100 ТБ данных, используя прямой ввод-вывод (без кэширования файлов ОС) и очень маленький блочный кэш размером 60 МБ. Разделение на разделы повышает производительность в 11 раз — с 5 до 55 операций в секунду.

```
./db_bench --benchmarks="чтение при записи[X3], статистика" --use_direct_reads=1 -
compaction_readahead_size 1048576 --use_existing_db --число= 2000000000 --
длительность 600 --размер кэша=62914560 -cache_index_and_filter_blocks=false
-статистика -гистограмма -bloom_bits=10 -target_file_size_base=268435456 -размер
блока=32768 -поток=32 -partition_filters -partition_indexes -index_per_partition 100
-pin_l0_filter_and_index_blocks_in_cache -benchmark_write_rate_limit 204800 -
max_bytes_for_level_base 134217728 -cache_high_pri_pool_ratio 0.9
```

🔗 Твердотельный накопитель, Linkbench

В этом примере у нас есть база данных размером 300 ГБ на SSD, и мы имитируем нехватку памяти, которая могла бы возникнуть при наличии других баз данных на том же узле, используя прямой ввод-вывод (без использования файлового кэша ОС) и блочный кэш размером 6 ГБ и 2 ГБ. Без разделения на разделы пропускная способность linkbench падает с 38 000 транзакций в секунду до 23 000 при уменьшении объема памяти с 6 ГБ до 2 ГБ. При разделении на разделы пропускная способность падает с 38 000 до 30 000 транзакций в секунду.

🔗 How to use it?

- `index_type = IndexType::kTwoLevelIndexSearch`
 - This is to enable partitioned indexes.
- `NewBloomFilterPolicy(BITS, false)`
 - Use full filters instead of block-based filter.
- `partition_filters = true`
 - This is to enable partitioned filters.
- `metadata_block_size = 4096`
 - This is the block size for index partitions.
- `cache_index_and_filter_blocks = false` [if you are on <= 5.14]
 - The partitions are stored in the block cache anyway. This is to control the location of top-level indexes (which easily fit into memory): pinned in heap or cached in the block cache. Having them stored in block cache is less experimented with.
- `cache_index_and_filter_blocks = true` and `pin_top_level_index_and_filter = true` [if you are on >= 5.15]
 - This would put everything in block cache but also pin the top-level indexes, which are quite small.
- `cache_index_and_filter_blocks_with_high_priority = true`
 - Recommended setting.
- `pin_l0_filter_and_index_blocks_in_cache = true`
 - Recommended setting as this property is extended to the index/filter partitions as well.
 - Use it only if the compaction style is level-based.
 - **Note:** with pinning blocks into the block cache, it could potentially go beyond the capacity if `strict_capacity_limit` is not set (which is the default case).
- block cache size: if you used to store the filter/index into heap, do not forget to increase the block cache size with the amount of memory that you are saving from the heap.

🔗 Current limitations

- Partitioned filters cannot be enabled without having partitioned index enabled as well.

- We have the same number of filter and index partitions. In other words, whenever an index block is cut, the filter block is cut as well. We might change it in future if it shows to be causing deficiencies.
- The filter block size is determined by when the index block is cut. We will soon extend `metadata_block_size` to enforce the maximum size on both filter and index blocks, i.e., a filter block is cut either when an index block is cut or when its size is about to exceed `metadata_block_size` (TODO).

🔗 Under the hood

Here we present the implementation details for the developers.

🔗 BlockBasedTable Format

You can study the BlockBasedTable format [here](#). With partitioning the difference would be that the index block

```
[index block]
```



is stored as

```
[index block - partition 1]
[index block - partition 2]
...
[index block - partition N]
[index block - top-level index]
```



and the footer of SST points to the top-level index block (which by itself is an index on index partition blocks). Each individual index block partition conforms the same format as `kBinarySearch`. The top-level index format also conforms with that of `kBinarySearch` and hence can be read using the normal data block readers.

Similar structure is used for partitioning the filter blocks. The format of each individual filter block conforms with that of `kFullFilter`. The top-level index format conforms with that of `kBinarySearch`, similar to top-level index of index blocks.

Note that with partitioning the SST inspection tools such `sst_dump` report the size of top-level index on index/filters rather than the collective size of index/filter blocks.

Builder

Partitioned index and filters are built by `PartitionedIndexBuilder` and `PartitionedFilterBlockBuilder` respectively.

`PartitionedIndexBuilder` maintains `sub_index_builder_`, a pointer to `ShortenedIndexBuilder`, to build the current index partition. When determined by `flush_policy_`, the builder saves the pointer along with the last key in the index block, and creates a new active `ShortenedIndexBuilder`. When `::Finish` is called on the builder, it calls `::Finish` on the earliest sub index builder and returns the resulting partition block. Next calls to `PartitionedIndexBuilder::Finish` will also include the offset of previously returned partition on the SST, which is used as the values of the top-level index. The last call to `PartitionedIndexBuilder::Finish` will finish the top-level index and return that instead. After storing the top-level index on SST, its offset will be used as the offset of the index block.

`PartitionedFilterBlockBuilder` inherits from `FullFilterBlockBuilder` which has a `FilterBitsBuilder` for building bloom filters. It also has a pointer to `PartitionedIndexBuilder` and invokes `ShouldCutFilterBlock` on it to determine when a filter block should be cut (right after when an index block is cut). To cut a filter block, it finishes the `FilterBitsBuilder` and stores the resulting block along with a partitioning key provided by `PartitionedIndexBuilder::GetPartitionKey()`, and reset the `FilterBitsBuilder` for the next partition. At the end each time `PartitionedFilterBlockBuilder::Finish` is invoked one of the partitions is returned, and also the offset of the previous partition is used to build the top-level index. The last call to `::Finish` will return the top-level index block.

The reasons for making `PartitionedFilterBlockBuilder` depend on `PartitionedIndexBuilder` was to enable an optimization for interleaving index/filter partitions on the SST file. That optimization not being pursued we are likely to cut this dependency in future.

Reader

Partitioned indexes are read via `PartitionIndexReader` which operates on the top-level index block. When `NewIterator` is invoked a `TwoLevelIterator` on the the top-level index block. This simple implementation is feasible since each index partition has `kBinarySearch` format which is the same format as data blocks, and thus can be easily plugged as the lower level iterator. If `pin_l0_filter_and_index_blocks_in_cache` is set, the lower level iterators are pinned to `PartitionIndexReader`, so their corresponding index partitions will be pinned in block cache as long as `PartitionIndexReader` is alive.

`BlockEntryIteratorState` uses a set of the pinned partition offsets to avoid unpinning an index partition twice.

`PartitionedFilterBlockReader` uses the top-level index to find the offset of the filter partition. It then invokes `GetFilter` on `BlockBasedTable` object to load the `FilterBlockReader` object on the filter partition from the block cache (or load it to the cache if it is not already there) and then releases the `FilterBlockReader` object. To extend `table_options.pin_l0_filter_and_index_blocks_in_cache` to filter partitions, `PartitionedFilterBlockReader` does not release the cache handles for such blocks (i.e., keep them pinned in block cache). It instead maintains `filter_cache_`, a map of pinned `FilterBlockReader`, which is also used to release the cache entries when `PartitionedFilterBlockReader` is destructed.

► Pages 167

Contents

- [RocksDB Wiki](#)
- [Overview](#)
- [RocksDB FAQ](#)
- [Terminology](#)
- [Requirements](#)
- [Contributors' Guide](#)
- [Release Methodology](#)
- [RocksDB Users and Use Cases](#)
- [RocksDB Public Communication and Information Channels](#)
- [Basic Operations](#)
 - [Iterator](#)
 - [Prefix seek](#)
 - [SeekForPrev](#)
 - [Tailing Iterator](#)
 - [Compaction Filter](#)
 - [Multi Column Family Iterator](#)
 - [Read-Modify-Write \(Merge\) Operator](#)
 - [Column Families](#)
 - [Creating and Ingesting SST files](#)
 - [Single Delete](#)
 - [SST Partitioner](#)
 - [Low Priority Write](#)
 - [Time to Live \(TTL\) Support](#)
 - [Transactions](#)
 - [Snapshot](#)
 - [DeleteRange](#)
 - [Atomic flush](#)
 - [Read-only and Secondary instances](#)
 - [Approximate Size](#)
 - [User-defined Timestamp](#)
 - [Wide Columns](#)

- [BlobDB](#)
- [Online Verification](#)
- **Options**
 - [Setup Options and Basic Tuning](#)
 - [Option String and Option Map](#)
 - [RocksDB Options File](#)
- [MemTable](#)
- [Journal](#)
 - [Write Ahead Log \(WAL\)](#)
 - [Write Ahead Log File Format](#)
 - [WAL Recovery Modes](#)
 - [WAL Performance](#)
 - [WAL Compression](#)
 - [MANIFEST](#)
 - [Track WAL in MANIFEST](#)
- **Cache**
 - [Block Cache](#)
 - [SecondaryCache \(Experimental\)](#)
- [Write Buffer Manager](#)
- [Compaction](#)
 - [Leveled Compaction](#)
 - [Universal compaction style](#)
 - [FIFO compaction style](#)
 - [Manual Compaction](#)
 - [Subcompaction](#)
 - [Choose Level Compaction Files](#)
 - [Managing Disk Space Utilization](#)
 - [Trivial Move Compaction](#)
 - [Remote Compaction](#)
- **SST File Formats**
 - [Block-based Table Format](#)
 - [PlainTable Format](#)
 - [CuckooTable Format](#)
 - [External Table](#)
 - [Index Block Format](#)
 - [Bloom Filter](#)
 - [Data Block Hash Index](#)
- [IO](#)
 - [Rate Limiter](#)
 - [SST File Manager](#)
 - [Direct I/O](#)
- [Compression](#)
 - [Dictionary Compression](#)
- [Full File Checksum and Checksum Handoff](#)
- [Background Error Handling](#)
- [Huge Page TLB Support](#)
- [Tiered Storage \(Experimental\)](#)
- **Logging and Monitoring**
 - [Logger](#)
 - [Statistics](#)
 - [Compaction Stats and DB Status](#)

- [Perf Context and IO Stats Context](#)
 - [EventListener](#)
- [Known Issues](#)
- [Troubleshooting Guide](#)
- [Tests](#)
 - [Stress Test](#)
 - [Fuzzing](#)
 - [Benchmarking](#)
- **Tools / Utilities**
 - [Administration and Data Access Tool](#)
 - [How to Backup RocksDB?](#)
 - [Replication Helpers](#)
 - [Checkpoints](#)
 - [How to persist in-memory RocksDB database](#)
 - [Third-party language bindings](#)
 - [RocksDB Trace, Replay, Analyzer, and Workload Generation](#)
 - [Block cache analysis and simulation tools](#)
 - [IO Tracer and Parser](#)
- **Implementation Details**
 - [Delete Stale Files](#)
 - [Partitioned Index/Filters](#)
 - [WritePrepared-Transactions](#)
 - [WriteUnprepared-Transactions](#)
 - [How we keep track of live SST files](#)
 - [How we index SST](#)
 - [Merge Operator Implementation](#)
 - [RocksDB Repairer](#)
 - [Write Batch With Index](#)
 - [Two Phase Commit](#)
 - [Iterator's Implementation](#)
 - [Simulation Cache](#)
 - [\[To Be Deprecated\] Persistent Read Cache](#)
 - [DeleteRange Implementation](#)
 - [unordered_write](#)
- **Extending RocksDB**
 - [RocksDB Configurable Objects](#)
 - [The Customizable Class](#)
 - [Object Registry](#)
- **RocksJava**
 - [RocksJava Basics](#)
 - [Logging in RocksJava](#)
 - [JNI Debugging](#)
 - [RocksJava API TODO](#)
 - [RocksJava Performance on Flash Storage](#)
 - [Tuning RocksDB from Java](#)
- **Performance**
 - [Performance Benchmarks](#)
 - [In Memory Workload Performance](#)
 - [Read-Modify-Write \(Merge\) Performance](#)
 - [Delete A Range Of Keys](#)

- [Write Stalls](#)
- [Pipelined Write](#)
- [MultiGet Performance](#)
- [Tuning Guide](#)
- [Memory usage in RocksDB](#)
- [Speed-Up DB Open](#)
- [Implement Queue Service Using RocksDB](#)
- [Asynchronous IO](#)
- [Off-peak in RocksDB](#)
- [Projects Being Developed](#)
- **Misc**
 - [Building on Windows](#)
 - [Developing with an IDE](#)
 - [Open Projects](#)
 - [Talks](#)
 - [Publication](#)
 - [Features Not in LevelDB](#)
 - [How to ask a performance-related question?](#)
 - [Articles about Rocks](#)

Clone this wiki locally

`https://github.com/facebook/rocksdb.wiki.git`

